

Solution 1

Solution 1 (a)

The function $\phi(x)$ returns the index j corresponding to the closest cluster center. Since there are k cluster centers indexed from 1 to k , the output is a single discrete integer.

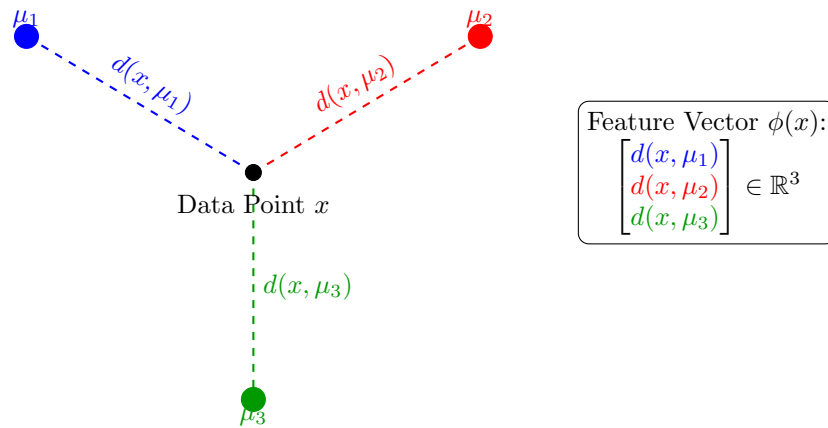
$\therefore$ Thus, the space is the discrete set of indices:

$$\phi(x) \in \{1, 2, \dots, k\}$$

Solution 1

Solution 1 (b)

Since distances in a metric space are non-negative real numbers, $\phi(x)$ is a vector in a k -dimensional Euclidean space (specifically the non-negative orthant).



$$\therefore \phi(x) \in \mathbb{R}^k$$

Solution 2

1. Case 1: Matrix D_1

From the matrix, the distances are:

$$d(x_1, x_2) = 1, \quad d(x_2, x_3) = 1, \quad d(x_1, x_3) = 2$$

Now, check the triangle inequality:

$$d(x_1, x_2) \leq d(x_1, x_2) + d(x_2, x_3)$$

$$d(x_1, x_2) = 1 \quad \text{and} \quad d(x_1, x_2) + d(x_2, x_3) = 1 + 1 = 2$$

From above, the triangle inequality holds.

We can construct these points in 1-dimensional Euclidean space ($\mathbb{R}^1$) by placing them on the number line:

$$x_1 = 0, \quad x_2 = 1, \quad x_3 = 2$$

2. Case 2: Matrix D_2

From the matrix, the distances are:

$$d(x_1, x_2) = 1, \quad d(x_2, x_3) = 1, \quad d(x_1, x_3) = 3$$

Now, check the triangle inequality:

$$d(x_1, x_3) \leq d(x_1, x_2) + d(x_2, x_3)$$

$$d(x_1, x_3) = 3 \quad \text{and} \quad d(x_1, x_2) + d(x_2, x_3) = 1 + 1 = 2$$

$$d(x_1, x_3) \not\leq d(x_1, x_2) + d(x_2, x_3)$$

$\therefore D_1$ is realizable ($x_1 = 0, x_2 = 1, x_3 = 2$) and D_2 is not realizable.

Solution 3

Solution 3 (a)

Let the points be denoted as vectors:

$$x = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \quad y = \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix}, \quad z = \begin{bmatrix} 0 \\ -2 \\ -4 \end{bmatrix}$$

The matrix D consists of squared Euclidean distances $D_{ij} = \|x_i - x_j\|^2$ or can be written as:

$$\begin{bmatrix} \|x_1 - x_1\|^2 & \|x_1 - y_1\|^2 & \|x_1 - z_1\|^2 \\ \|x_2 - x_1\|^2 & \|y_2 - y_2\|^2 & \|x_2 - z_2\|^2 \\ \|x_3 - x_1\|^2 & \|x_1 - y_1\|^2 & \|x_3 - z_3\|^2 \end{bmatrix}$$

Now calculate each distance element:

1. D_{11}

$$\|x - x\|^2 = (1 - 1)^2 + (2 - 2)^2 + (3 - 3)^2 = 0^2 + 0^2 + 0^2 = 0$$

2. D_{12}

$$\|x - y\|^2 = (1 - (-1))^2 + (2 - 0)^2 + (3 - 1)^2 = 2^2 + 2^2 + 2^2 = 4 + 4 + 4 = 12$$

3. D_{13}

$$\|x - z\|^2 = (1 - 0)^2 + (2 - (-2))^2 + (3 - (-4))^2 = 1^2 + 4^2 + 7^2 = 1 + 16 + 49 = 66$$

4. D_{21}

$$\|y - x\|^2 = (-1 - 1)^2 + (0 - 2)^2 + (1 - 3)^2 = (-2)^2 + (-2)^2 + (-2)^2 = 4 + 4 + 4 = 12$$

5. D_{22}

$$\|y - y\|^2 = (-1 - (-1))^2 + (0 - 0)^2 + (1 - 1)^2 = 0^2 + 0^2 + 0^2 = 0$$

6. D_{23}

$$\|y - z\|^2 = (-1 - 0)^2 + (0 - (-2))^2 + (1 - (-4))^2 = (-1)^2 + 2^2 + 5^2 = 1 + 4 + 25 = 30$$

7. D_{31}

$$\|z - x\|^2 = (0 - 1)^2 + (-2 - 2)^2 + (-4 - 3)^2 = (-1)^2 + (-4)^2 + (-7)^2 = 1 + 16 + 49 = 66$$

8. D_{32}

$$\|z - y\|^2 = (0 - (-1))^2 + (-2 - 0)^2 + (-4 - 1)^2 = 1^2 + (-2)^2 + (-5)^2 = 1 + 4 + 25 = 30$$

9. D_{33}

$$\|z - z\|^2 = (0 - 0)^2 + (-2 - (-2))^2 + (-4 - (-4))^2 = 0$$

$$\therefore D = \begin{bmatrix} 0 & 12 & 66 \\ 12 & 0 & 30 \\ 66 & 30 & 0 \end{bmatrix}$$

Solution 3

Solution 3 (b)

H is the centering matrix defined as:

$$H = I - \frac{1}{n} \mathbf{1}\mathbf{1}^T$$

,

Let, $n = 3$ and I is the identity matrix. Now, calculate H

$$H = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} - \frac{1}{3} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 \end{bmatrix}$$

$$H = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} - \begin{bmatrix} 1/3 & 1/3 & 1/3 \\ 1/3 & 1/3 & 1/3 \\ 1/3 & 1/3 & 1/3 \end{bmatrix}$$

$$\therefore H = \begin{bmatrix} 2/3 & -1/3 & -1/3 \\ -1/3 & 2/3 & -1/3 \\ -1/3 & -1/3 & 2/3 \end{bmatrix}$$

Solution 3

Solution 3 (c)

This operation converts the squared distance matrix into the centered Gram matrix B .

$$B = -\frac{1}{2}HDH$$

Performing the matrix multiplication: 1. $DH = \begin{bmatrix} 0 & 12 & 66 \\ 12 & 0 & 30 \\ 66 & 30 & 0 \end{bmatrix} \begin{bmatrix} 2/3 & -1/3 & -1/3 \\ -1/3 & 2/3 & -1/3 \\ -1/3 & -1/3 & 2/3 \end{bmatrix} = \begin{bmatrix} -26 & -14 & 40 \\ -2 & 14 & -12 \\ 34 & -12 & -22 \end{bmatrix}$

2. $H(DH) = \begin{bmatrix} 2/3 & -1/3 & -1/3 \\ -1/3 & 2/3 & -1/3 \\ -1/3 & -1/3 & 2/3 \end{bmatrix} \begin{bmatrix} -26 & -14 & 40 \\ -2 & 14 & -12 \\ 34 & -12 & -22 \end{bmatrix} = \begin{bmatrix} -28 & -4 & 32 \\ -4 & -4 & 8 \\ 32 & 8 & -40 \end{bmatrix}$

3. Multiply by $-\frac{1}{2}$:

$$B = -\frac{1}{2} \begin{bmatrix} -28 & -4 & 32 \\ -4 & -4 & 8 \\ 32 & 8 & -40 \end{bmatrix}$$

$$\therefore \begin{bmatrix} 14 & 2 & -16 \\ 2 & 2 & -4 \\ -16 & -4 & 20 \end{bmatrix}$$

Solution 3

Solution 3 (d)

The definition of the Gram matrix for a set of data vectors (arranged as rows in X) is $G = XX^T$. This means the entry G_{ij} is the dot product of the i -th point and the j -th point.

Let the data matrix X be defined as:

$$X = \begin{bmatrix} 1 & 2 & 3 \\ -1 & 0 & 1 \\ 0 & -2 & -4 \end{bmatrix}$$

The rows correspond to vectors x, y, z . We calculate the dot products directly:

$$x \cdot x = (1)(1) + (2)(2) + (3)(3) = 1 + 4 + 9 = 14$$

$$y \cdot y = (-1)(-1) + (0)(0) + (1)(1) = 1 + 0 + 1 = 2$$

$$z \cdot z = (0)(0) + (-2)(-2) + (-4)(-4) = 0 + 4 + 16 = 20$$

$$x \cdot y = (1)(-1) + (2)(0) + (3)(1) = -1 + 0 + 3 = 2$$

$$x \cdot z = (1)(0) + (2)(-2) + (3)(-4) = 0 - 4 - 12 = -16$$

$$y \cdot z = (-1)(0) + (0)(-2) + (1)(-4) = 0 + 0 - 4 = -4$$

By symmetry, $x \cdot y = y \cdot x$, etc. Constructing the matrix from these values:

$$XX^T = \begin{bmatrix} 14 & 2 & -16 \\ 2 & 2 & -4 \\ -16 & -4 & 20 \end{bmatrix}$$

This result matches the matrix B calculated in part (c) exactly. *Note: The operation in (c) produces the Gram matrix of the centered data. Since the mean of the original points is $\vec{0}$, the centered Gram matrix and the original Gram matrix are identical.*

$\therefore$ Yes, the result matches the matrix from part (c).

Solution 4

Solution 4 (a)

Let the points be vectors $x_1, \dots, x_n$. The definition of **orthonormal** is: 1. Orthogonal: The dot product of distinct vectors is zero ($x_i^T x_j = 0$ for $i \neq j$). 2. Normal: The length of each vector is one ($x_i^T x_i = \|x_i\|^2 = 1$).

The Gram matrix G is defined as $G_{ij} = x_i^T x_j$. Applying the definition:

- Diagonal entries ($i = j$): $G_{ii} = x_i^T x_i = 1$.
- Off-diagonal entries ($i \neq j$): $G_{ij} = x_i^T x_j = 0$.

Thus, G is the Identity matrix.

$\therefore G = I_n$ (The $n \times n$ Identity Matrix)

Solution 4

Solution 4 (b)

The squared Euclidean distance between two vectors x_i and x_j is given by the expansion:

$$\begin{aligned} \|x_i - x_j\|^2 &= (x_i - x_j)^T (x_i - x_j) \\ &= x_i^T x_i - 2x_i^T x_j + x_j^T x_j \end{aligned}$$

Substitute the orthonormal properties from part (a):

- If $i = j$: $\|x_i - x_i\|^2 = 0$.
- If $i \neq j$: $\|x_i - x_j\|^2 = 1 - 2(0) + 1 = 2$.

Therefore, the squared distance matrix D has 0 on the diagonal and 2 everywhere else.

$$\therefore D = 2(\mathbf{1}\mathbf{1}^T - I) = \begin{bmatrix} 0 & 2 & \dots & 2 \\ 2 & 0 & \dots & 2 \\ \vdots & \vdots & \ddots & \vdots \\ 2 & 2 & \dots & 0 \end{bmatrix}$$

Solution 5

Solution 5 (a)

Python Code

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.metrics import euclidean_distances
4
5 def cmdscale(D_squared, k=2):
6     """ implement cMDS from lecture """
7     n = D_squared.shape[0]
8
9     # calc centering matrix H
10    H = np.eye(n) - (1/n) * np.ones((n, n))
11
12    # calc B = -1/2 * H * D_squared * H
13    B = -0.5 * H @ D_squared @ H
14
15    # spectral decomposition
16    # note: np.linalg.eigh returns eigenvalues in ascending order
17    evals, evects = np.linalg.eigh(B)
18
19    # sort
20    idx = np.argsort(evals)[::-1]
21    evals = evals[idx]
22    evects = evects[:, idx]
23
24    # 4. zero negative eigenvalues (for non-Euclidean noise)
25    evals_pos = np.maximum(evals, 0)
26
27    # calc coordinates Y
28    Lambda_sqrt = np.diag(np.sqrt(evals_pos))
29
30    # project: Y = U * Lambda^1/2
31    Y = evects @ Lambda_sqrt
32
33    # return first k columns
34    return Y[:, :k]
35
36 # read data
37 raw_data = open('cities.txt').read().split()
38 city_names = raw_data[2:12]
39
40 # massage data
41 matrix_values = [float(x) for x in raw_data[12:]]
42 D = np.array(matrix_values).reshape(10, 10)
43 D_squared = D ** 2
44
45 # run cmdscale(D_squared, k=2) function
46 Y_k = cmdscale(D_squared, k=2)
47
48 def rotate(points, angle_degrees):
49     theta = np.radians(angle_degrees)
50     c, s = np.cos(theta), np.sin(theta)
51     R = np.array([[c, -s], [s, c]])
52     return points @ R.T
53
54 # rotate 180
55 Y_rotated = rotate(Y_k, angle_degrees=180)
56
57 # plot
58 plt.figure(figsize=(10, 6))
59 plt.scatter(Y_rotated[:, 0], Y_rotated[:, 1], c='blue', s=100)
60 for i, name in enumerate(city_names):
61     plt.annotate(name, (Y_rotated[i, 0], Y_rotated[i, 1]),
62                 xytext=(5, 5), textcoords='offset points')
63

```

```
64 plt.title("Reconstructed Map from Distance Matrix (MDS)")
65 plt.grid(True)
66 plt.axis('equal')
67 plt.savefig("q5a.png")
```

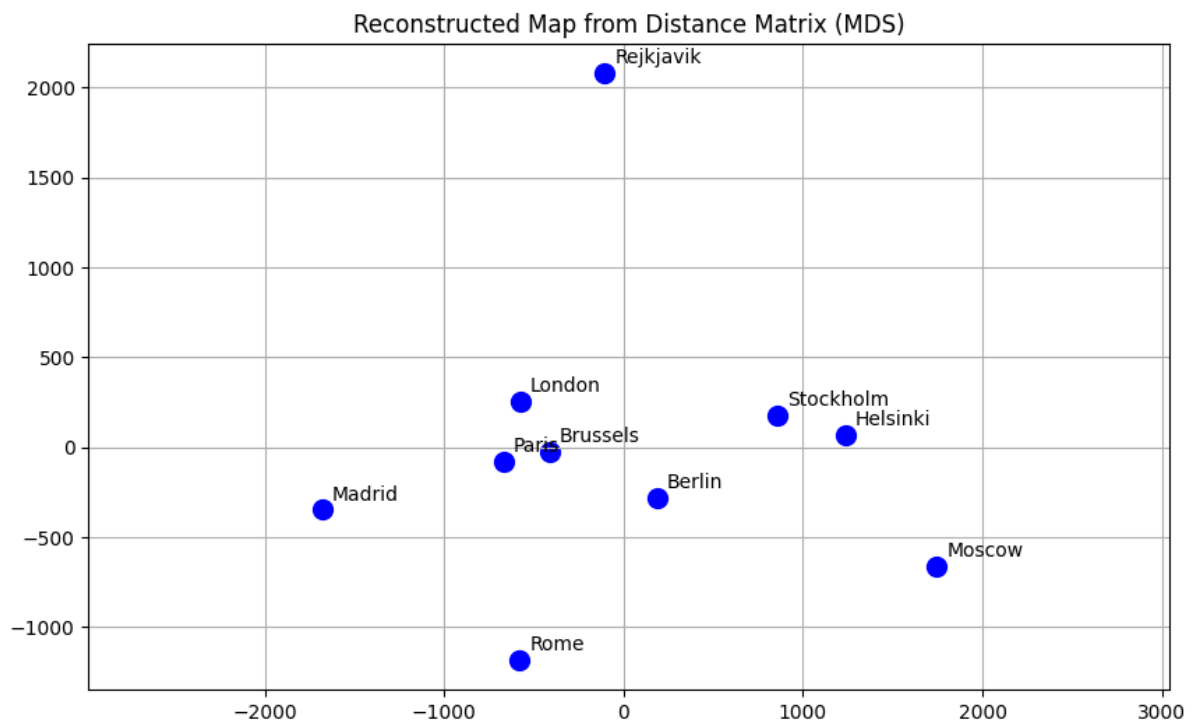
Plot

Figure 1: Plot of cities.txt

Solution 5

Solution 5 (b)

Python Code

```

1  def cmdscale(D, k=2):
2      """classicMDS: Projects n points into k-dimensional space."""
3      n = D.shape[0]
4      H = np.eye(n) - (1/n) * np.ones((n, n))
5      B = -0.5 * H @ D @ H
6
7      evals, evects = np.linalg.eigh(B)
8
9      # sort
10     idx = np.argsort(evals)[::-1]
11     evals = evals[idx]
12     evects = evects[:, idx]
13
14     # project
15     w_pos = np.maximum(evals, 0)
16     Y = evects @ np.diag(np.sqrt(w_pos))
17     return Y[:, :k]
18
19 def rotate_data(X, angle_degrees):
20     """Rotate 2D data X by angle_degrees (counter-clockwise)"""
21     theta = np.radians(angle_degrees)
22     c, s = np.cos(theta), np.sin(theta)
23
24     # define rotation matrix
25     R = np.array([[c, -s], [s, c]])
26
27     # apply R to every point
28     return X @ R.T
29
30 # plot
31 def plot_map(X, labels, title="MDS Map"):
32     plt.figure(figsize=(8, 8))
33     plt.scatter(X[:, 0], X[:, 1], c='red', marker='o')
34     for i, txt in enumerate(labels):
35         plt.annotate(txt, (X[i, 0], X[i, 1]), xytext=(5, 5), textcoords='offset points')
36
37     plt.title(title)
38     plt.grid(True)
39     plt.axis('equal')
40     plt.show()
41
42 # get raw MDS coordinates
43 Y_raw = cmdscale(D_squared)
44
45 # rotate
46 best_angle = 190
47 Y_rotated = rotate_data(Y_raw, best_angle)
48
49 # call plot_map function
50 plot_map(Y_rotated, city_names, title=f"European Cities (Rotated {best_angle} degrees)")

```

Plot

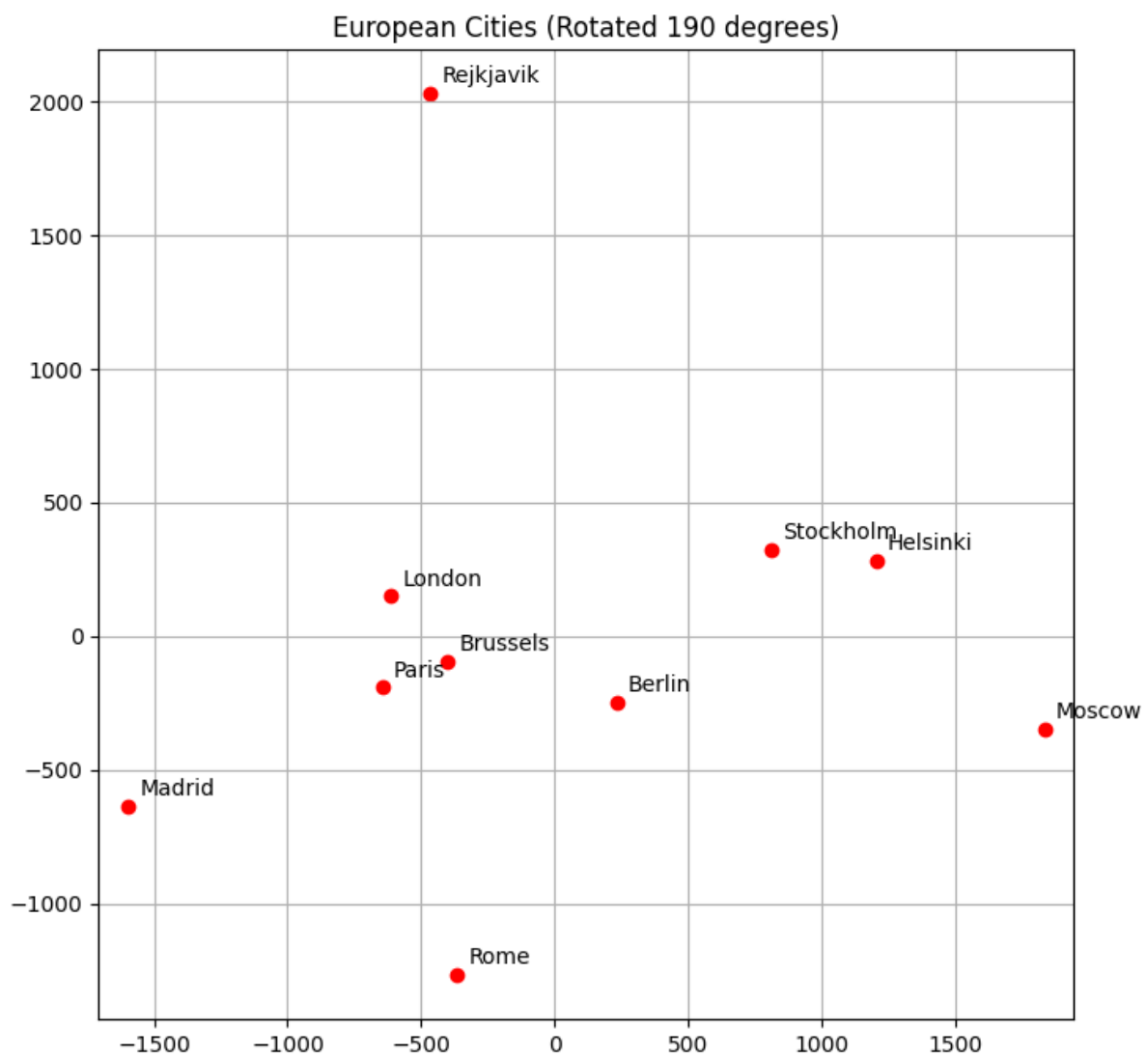


Figure 2: Plot 5b

Solution 5

Solution 5 (c)

Python Code

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.manifold import MDS
4 # sklearn MDS
5 mds = MDS(n_components=2, dissimilarity='precomputed', random_state=42,
6           normalized_stress='auto')
7 Y_sklearn = mds.fit_transform(D) #
8 # rotation
9 def rotate_data(X, angle_degrees):
10     theta = np.radians(angle_degrees)
11     c, s = np.cos(theta), np.sin(theta)
12     R = np.array([[c, -s], [s, c]])
13     return X @ R.T
14
15 def plot_embedding(X, title, ax):
16     ax.scatter(X[:, 0], X[:, 1], c='green', s=100)
17     for i, txt in enumerate(city_names):
18         ax.annotate(txt, (X[i, 0], X[i, 1]), xytext=(5, 5), textcoords='offset points')
19     ax.set_title(title)
20     ax.grid(True)
21     ax.axis('equal')
22
23 # find correct rotation
24 angle = 150
25 Y_rotated = rotate_data(Y_sklearn, angle)
26 # plot
27 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 8))
28 plot_embedding(Y_sklearn, "Sklearn MDS (Raw Output)", ax1)
29 plot_embedding(Y_rotated, f"Sklearn MDS (Rotated {angle} degrees)", ax2)
30 plt.savefig("q5c.png")

```

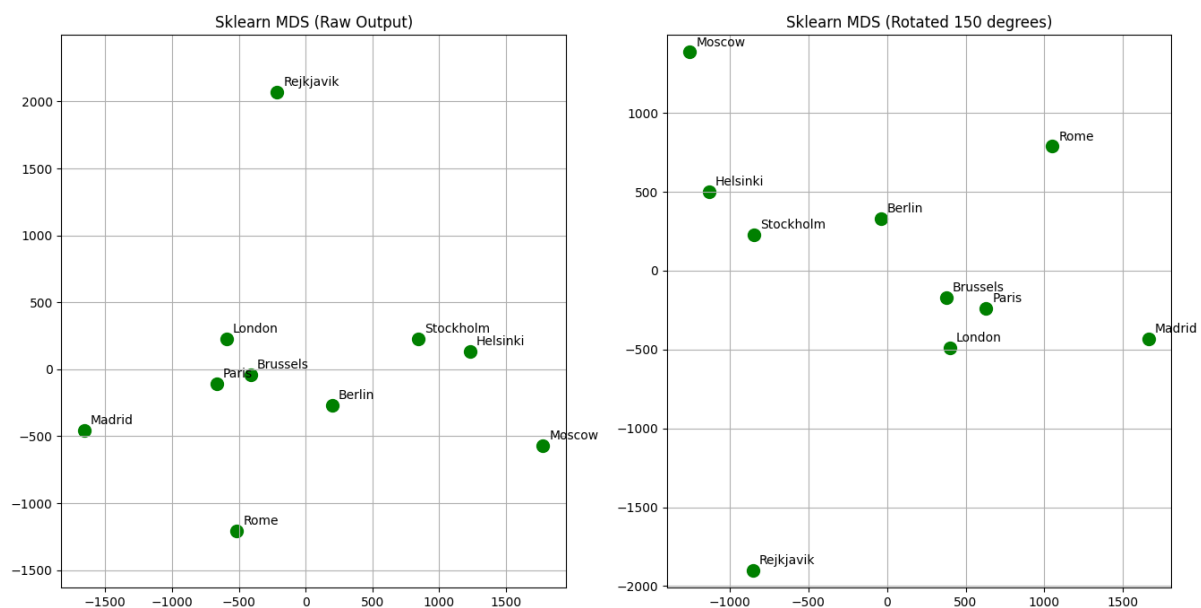


Figure 3: Plot 5 c